Note that the solution we discussed above is task-specific and data set-specific. We knew that the degree of lexical overlap was confusing the classifier. This allowed us to provide counter-examples which overcame this bias in the data. So how can we figure out what kind of data set biases are confusing the classifier? One way to know your data set is to compute Pointwise Mutual Information between the labels and the words in the sentences pertaining to that class label, and do a manual inspection. This will allow you to figure out whether any words that are unrelated to the label are associated spuriously with it. For example, if the word 'Trump' was always associated with negative samples, the classifier can learn a spurious cue that 'Trump' is a sentiment-bearing word when it actually is not so.

Analysing the Pointwise Mutual Information between the words in the vocabulary and the class label allows you to see if there is a high degree of correlation between certain words and the label, which was NOT actually intended. If there is such spurious correlation, you can then add examples which counter this effect. This idea of adding counter examples is described in the paper 'PAWS - Paraphrase Adversaries from Word Scrambling'.

Another way to understand the reasons why your classifier fails to predict a simple example correctly is to use an interpretability library, which can map back the decision to the words that influenced the decision. A popular interpretability library is LIME. One thing to remember when using LIME is that it only explains the decision of each specific example and does not actually provide any global explanations.

Even if your data set did not have any biases, the latter can still creep in and your classifier can become a victim to them. What are the other sources of these biases? If you are using pretrained word embeddings in your application, your classifier can be vulnerable to the biases in the embeddings. For example, it has been shown that many of the pretrained word embeddings exhibit gender bias. The word 'nurse' is closer in the word embedding space to 'she' than 'he'. Similarly, the word 'doctor' is closer in the word embedding space to 'he' than 'she'.

If you try the analogy test, "Man is to computer programmer as woman is to X," on any of the pretrained word embeddings, the answer is highly likely to be 'home maker'. The data on which pretrained word embeddings are based already has human-induced bias. This bias gets picked up and amplified by the classifier that uses the embeddings for the input representation.

Consider a classifier that can classify resumes for a job as appropriate or inappropriate. If the classifier was using pretrained word embeddings, which exhibit gender bias, it is likely to give a higher score to the resumes of men rather than women, for a computer programmer's job. This will be the case even when the classifier is NOT explicitly using the gender attribute in its classification decisions. Such hidden biases are difficult to understand and identify. Even if you don't include any specific 'unfair' features in your classifier, there can be proxy 'unfair' features. For example, a ZIP code feature can serve as a proxy for race, depending on whether certain ZIP codes are strongly correlated with certain races.

Fairness in machine learning is a complex but well studied problem. A good overview can be found in the Google developers' machine learning section. One way to overcome the classification issues with biases is to conduct testing for different input combinations so that the input space gets covered adequately. Of course, this comes back to the question of how much data is sufficient to cover the input space adequately.

Biases in data sets and embeddings propagate to classifiers. They can lead to incorrect classification decisions, which can affect performance or discriminate against a set of individuals or groups. The problem is inherent to the machine learning paradigm, since correlation driven learning makes the classifier prone to learning statistical biases. Instead of worrying about increasing the training data set size, we need to start considering the diversity and richness of the data set. Data scientists need to understand their data sets fairly well. If the data set has surface level statistical biases, targeted counter examples that can de-bias the data set need to be added. Blindly adding data will not help in de-biasing the data set.

Another way of adding the requisite samples to training data is to use active learning methods. In active learning, the classifier uses a heuristic to decide among a set of samples, and chooses one that it will pass on to an oracle to get its ground truth label. The idea behind active learning is that since the sample chosen is the one that gives the most information to a classifier in reducing its uncertainty, it will help improve classifier performance. Typically, a human in the loop is used as the oracle in the active learning setup. Choosing a sample for active learning is typically classifier internal dependent. A typical heuristic is to use the samples that the classifier has the least confidence about with regard to its classification decision. By learning them correctly, it can correct its classification boundary appropriately. However, there are cases where the classifier makes the wrong classification decision with high confidence. Active learning cannot help such cases. We will discuss how to deal with this problem in our next column.

If you have any favorite programming questions/software topics that you would like to discuss on this forum, please send them to me, along with your solutions and feedback, at *sandyasm_AT_yahoo_DOT_com*. Wishing all our readers happy coding until next month! END

By: Sandya Mannarswamy

The author is an expert in natural language processing and is currently working as an independent researcher. Her interests include natural language processing, machine learning and AI.